AKILABANA MOKILI Joseph
2ère année de BTS SIO - Services Informatiques aux Organisations

# Note de synthèse

du 12 Janvier  au 20 Février 2026

Réalisé par AKILABANA Joseph

# Sommaire

Présentation.....2

Organigramme.....5

Projets (Docker).....6
- Introduction.....6
- Installation et lancement de Docker.....7
- Installation de conteneur MariaDB.....8
- Création des conteneurs Mariadb (Maître-Esclave).....8
- Synchronisation des conteneurs.....10
- Création des conteneurs Mariadb (Maître-Esclave).....12
- Création des 2 base de donnée et réplication.....14

Projet (Kubernetes).....15
- Présentation.....15
- Création des manifesté (Mariadb et phpmyadmin).....16
- Reverse proxy des manifestes (MariaDB et phpMyAdmin).....17

Projet Formulaire d'inscription W3TEL.....19
- Commandes utilisées et explication.....19
- Démarche de conception (partie front-end).....21
- Création de formulaire avec Vue.js.....22
- Architecture générale de l'application.....23
- Authentification et sécurité.....24
- Gestion des utilisateurs et suivi des connexions.....25
- Tableau de bord (Dashboard.vue).....26
- Gestion des serveurs SSH.....27
- Terminal SSH en temps réel.....27
- Gestion de la navigation et des accès.....29

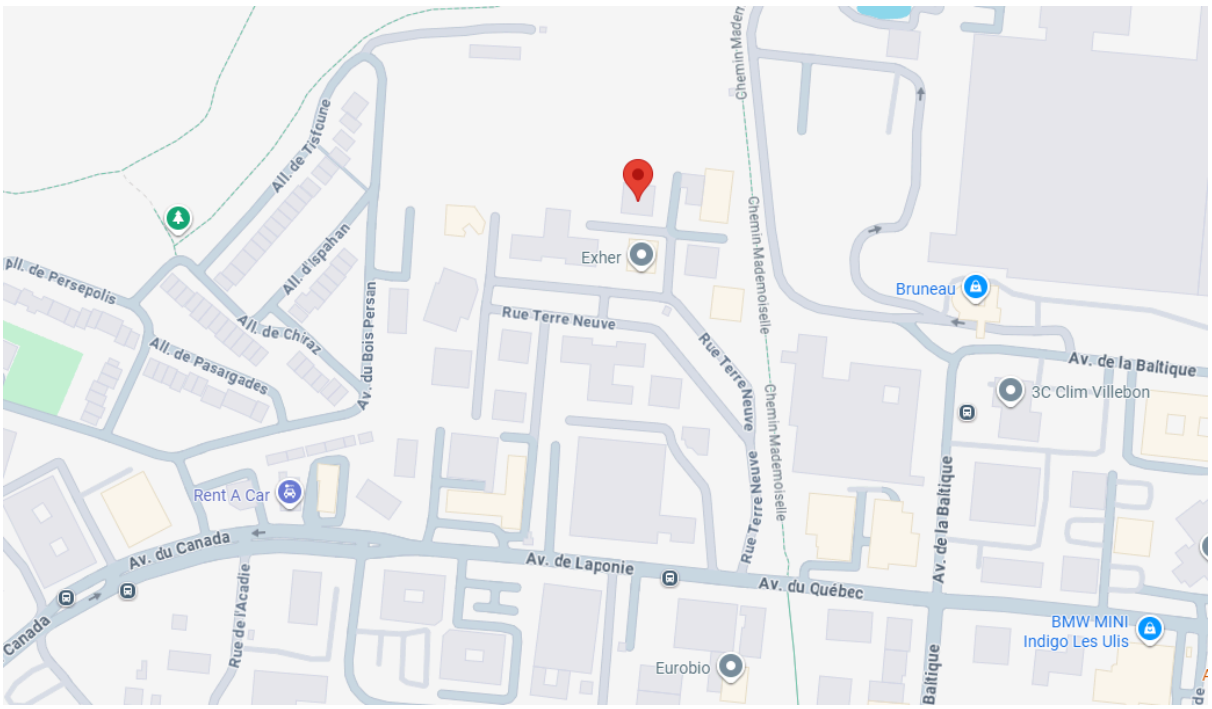
Conclusion.....30

Annexes.....31
- Annexes 1.....31
- Annexes 2 (Fin de travail Kubernetes).....32
- Annexes 3 (partie front-end).....33
- Annexes 3 (partie back-end).....33

Glossaire.....34

# Présentation

W3TEL est une entreprise française spécialisée dans les télécommunications et les services de communication pour les entreprises.
Créée en 2005 et basée aux Ulis (91), elle propose des solutions complètes allant de la téléphonie fixe et mobile à l'accès Internet haut débit, en passant par des services de visioconférence et communication unifiée.

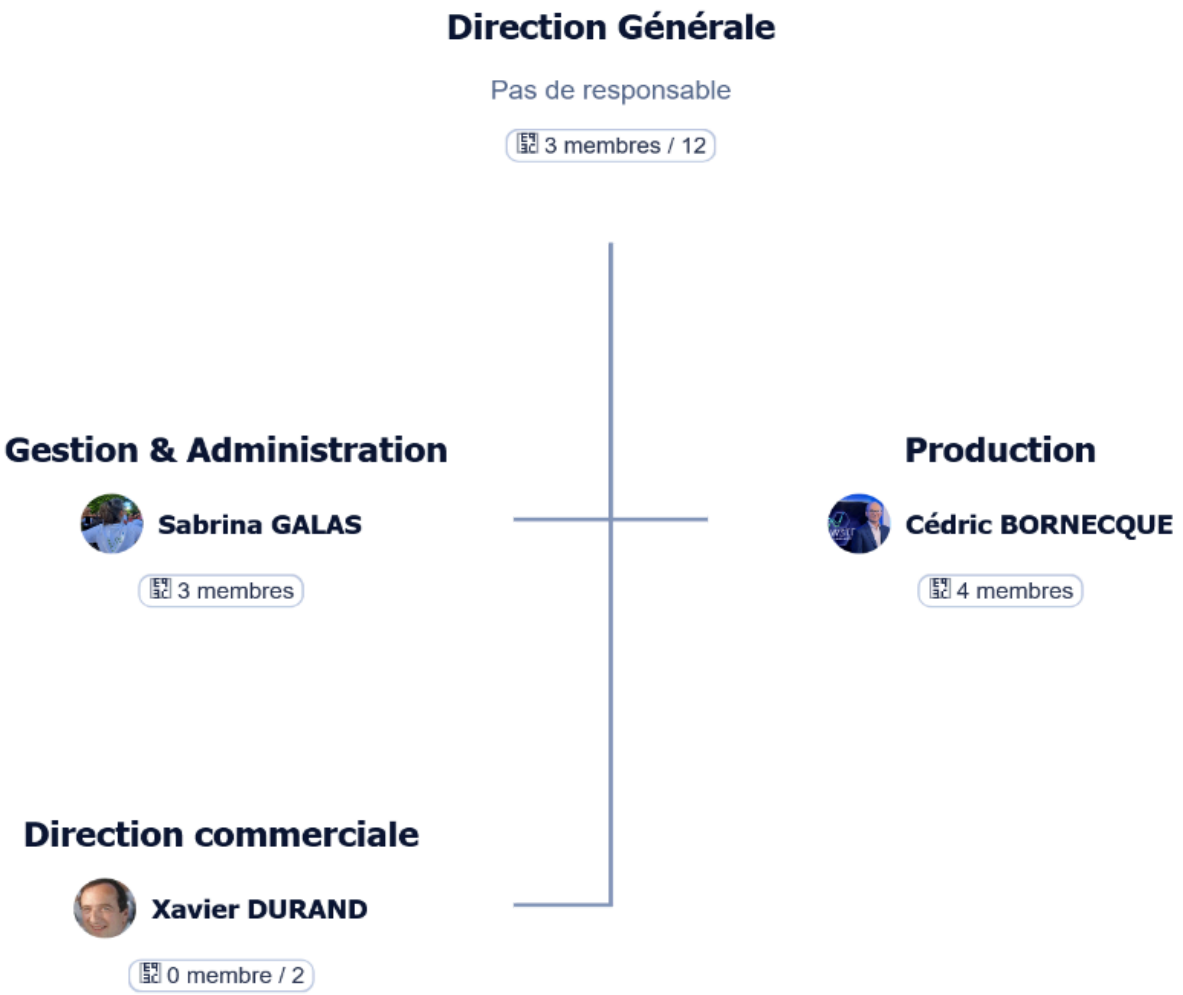



L'entreprise se distingue par la construction de sa propre infrastructure technique, ce qui lui permet d'offrir des services stables, sécurisés et performants, tout en assurant un service client réactif et de qualité. Avec une équipe restreinte mais expérimentée, W3TEL cible principalement les PME et organisations ayant des besoins avancés en communication, et collabore avec des partenaires technologiques pour renforcer ses capacités et garantir la conformité réglementaire. Son chiffre d'affaires avoisine 1,9 million d'euros, reflétant une activité stable et un positionnement solide sur le marché français des solutions de communication d'entreprise.

Cet établissement public possède structure d’activités particulier :

- **L’équipe de support technique et réseaux** dirigée par Cédric et comme membres il y a **Jean Baptiste**,**Sara** qui qui s’occupe de la partie (réseaux et cybersécurité) et Aubin (mon tuteur) qui est le responsable de système réseaux et sécurité .
- **L’équipe d’administrateur, comptabilité** s'occupe de recrutement ou encore que l’entreprise soit conforme au lois et au sein de cette équipe il ya 2 membres **Sabrina** et **Tania**
- **L’équipe de marketing** avec comme membre Emilie qui là pour mettre en valeur les produit de la l’entreprise en utilisant plusieurs par exemple les réseaux sociaux.
- **L’équipe de Commerciaux,** leur objectif est de vendre les produits et mettre en avant les produits par certains acheteurs potentiels.

# Organigramme



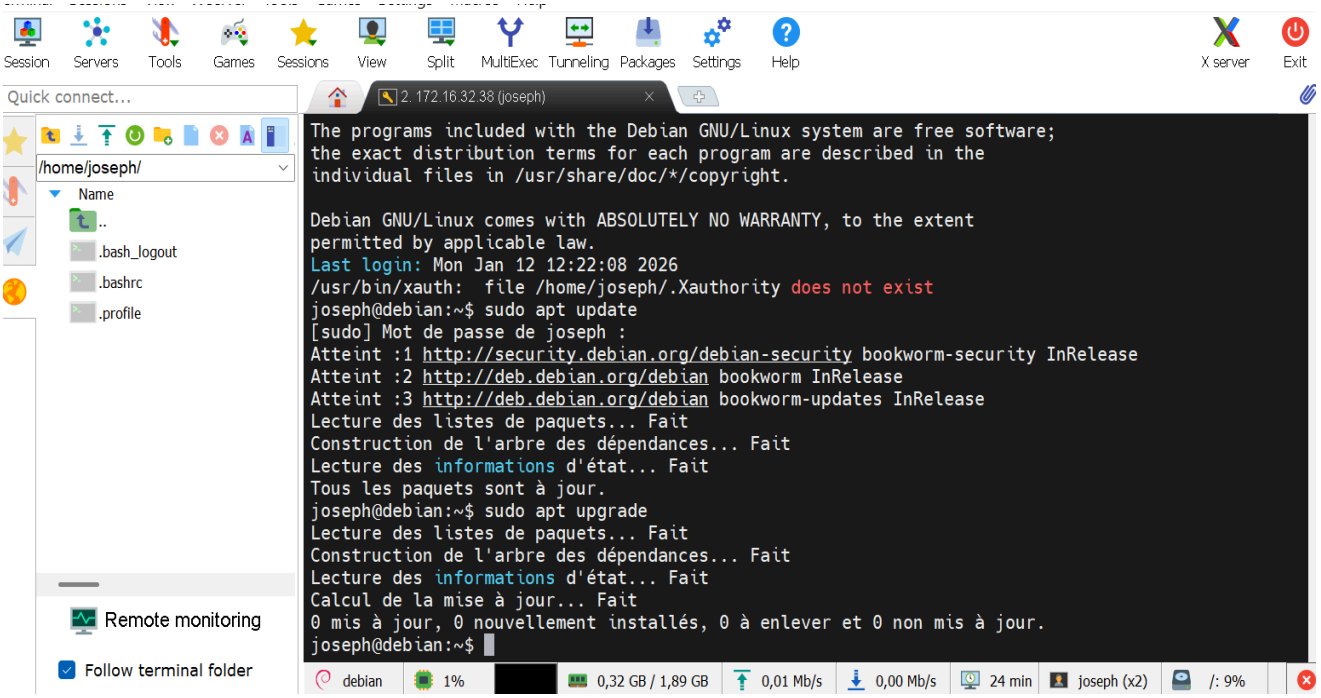

Aubin KUSONDAMA est non seulement le responsable d'administrateur réseau de W3TEL , mais également la personne qui s'occupe directement de moi. Il cumule plusieurs rôles opérationnels au sein de l'entreprise et fait partie de l'équipe de production dirigée par Cédric BORNECQUE.
Il prête main forte à l'équipe de production car il excelle dans plusieurs domaines.

# Projets (Docker)

## Introduction

Dans le cadre de ma deuxième année de BTS Services Informatiques aux Organisations (option SLAM), Lors de la première semaine de stage, Monsieur Aubin KUSONDAMA me confie la tâche qui consiste à réaliser un projet : déployer un serveur de base de données MariaDB dans des conteneurs, à créer et synchroniser plusieurs bases de données, puis à assurer leur réplication dans un environnement maîtrisé. Ce projet avait pour objectif de me permettre d'acquérir et de renforcer des compétences techniques essentielles, notamment en conteneurisation, en administration de bases de données et en orchestration de services.

Il visait également à reproduire des situations proches de celles rencontrées en milieu professionnel, telles que la gestion de la persistance des données, la synchronisation entre bases et le déploiement d'applications dans un environnement scalable et sécurisé.



## Installation et lancement de Docker

La première étape du projet a consisté à installer Docker sur la machine Linux et à vérifier son bon fonctionnement à l'aide des commandes de base telles que `docker --version` et `docker ps`. Une fois Docker opérationnel, les images nécessaires ont été récupérées depuis Docker Hub, notamment l'image officielle de MariaDB, afin de pouvoir créer et exécuter les conteneurs.

```
joseph@debian:~$ sudo systemctl start docker
joseph@debian:~$ sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable docker
joseph@debian:~$ docker --version
Docker version 29.1.4, build 0e6fee6
joseph@debian:~$ sudo usermod -aG docker $USER
joseph@debian:~$ newgrp docker
joseph@debian:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:d4aaab6242e0cace87e2ec17a2ed3d779d18fbfd03042ea58f2995626396a274
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
```

```
joseph@debian:~$ sudo docker run -d \
  --name mariadb-container \
  -e MYSQL_ROOT_PASSWORD=joseph \
  -p 3306:3306 \
  -v mariadb_data:/var/lib/mysql \
  mariadb:latest
dd90de1e0fcdfb14f511b8e9e50aff5a254d3c2f87cd4bf3920d5480a6410808
joseph@debian:~$ sudo docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED          STATUS          PORTS                                       N
AMES
dd90de1e0fcd   mariadb:latest   "docker-entrypoint.s…"   31 seconds ago   Up 30 seconds   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   m
ariadb-container
joseph@debian:~$ sudo docker exec -it mariadb-container mariadb -uroot -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 12.1.2-MariaDB-ubu2404 mariadb.org binary distribution
```

## Installation de conteneur MariaDB

Un premier conteneur MariaDB a été déployé afin de valider le bon fonctionnement du service de base de données. Les variables d'environnement nécessaires, comme le mot de passe root, ont été configurées, et l'accès à la base a été testé à l'aide du client MariaDB. Cette étape a permis de vérifier que le service MariaDB démarrait correctement dans un conteneur.

```
joseph@debian:~$ sudo docker run -d \
  --name mariadb-container \
  -e MYSQL_ROOT_PASSWORD=monpassword \
  -p 3306:3306 \
  -v mariadb_data:/var/lib/mysql \
  mariadb:latest
[sudo] Mot de passe de joseph :
Unable to find image 'mariadb:latest' locally
latest: Pulling from library/mariadb
```

## Création des conteneurs Mariadb (Maître-Esclave)

```
joseph@debian:~$ sudo docker run -d \
  --name mariadb-container \
  -e MYSQL_ROOT_PASSWORD=monpassword \
  -p 3306:3306 \
  -v mariadb_data:/var/lib/mysql \
  mariadb:latest
[sudo] Mot de passe de joseph :
Unable to find image 'mariadb:latest' locally
latest: Pulling from library/mariadb
ba2d27f4ceca: Pull complete
20043066d3d5: Pull complete
75e5c9f5eeb0: Pull complete
eca7ec75082e: Pull complete
4be15142d46e: Pull complete
9047f410cb7d: Pull complete
be922ac1f9ed: Pull complete
ee5b64c3e2f5: Pull complete
0711efb1517f: Download complete
c2fd0cca54b4: Download complete
Digest: sha256:e1bcd6f85781f4a875abefb11c4166c1d79e4237c23de597bf0df81fec225b40
Status: Downloaded newer image for mariadb:latest
52d59d3049c394db058391bc20f6fe8d13cf7bd131edfe89fe2ac8b3ed3a4e6b
joseph@debian:~$ sudo docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED         STATUS         PORTS                                       NAM
ES
52d59d3049c3   mariadb:latest   "docker-entrypoint.s…"   2 minutes ago   Up 2 minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   mar
iadb-container
joseph@debian:~$ sudo docker exec -it mariadb-container mysql -uroot -p
```

Dans la continuité du projet, le fonctionnement d'une architecture Maître-Esclave a été étudié afin de mieux comprendre l'organisation d'une base de données répartie. Cette architecture repose sur deux conteneurs MariaDB ayant des rôles distincts. Le conteneur maître est utilisé pour effectuer les ajouts et les modifications des données, tandis que le conteneur esclave permet principalement de consulter les données et d'assurer leur copie. Même si la réplication automatique complète de MariaDB n'a pas été mise en place à ce stade, cette partie du projet a permis de comprendre l'utilité de cette architecture, notamment pour améliorer la disponibilité des données et renforcer la fiabilité du système.

Par la suite, j'ai mis en place deux conteneurs MariaDB distincts : l'un jouant le rôle de maître et l'autre celui d'esclave. Le conteneur maître a été configuré pour enregistrer les modifications grâce aux logs binaires et pour disposer d'un identifiant propre (`server-id`). De son côté, le conteneur esclave a été paramétré pour se connecter automatiquement au maître à l'aide d'un utilisateur spécialement créé pour la réplication. Cette étape a permis d'établir une relation claire entre les deux bases et de préparer le fonctionnement de la réplication maître-esclave (voir Annexes 1).

## Synchronisation des conteneurs

Une fois les deux conteneurs MariaDB opérationnels, la synchronisation entre le maître et l'esclave a été mise en œuvre afin d'assurer la cohérence des données. L'esclave a été configuré pour se connecter au maître et lire en continu les journaux binaires générés par celui-ci. Chaque modification effectuée sur la base du maître (création de tables, insertion, mise à jour ou suppression de données) était ainsi automatiquement transmise et appliquée sur l'esclave, sans intervention manuelle.

Afin de valider le bon fonctionnement de cette synchronisation, des commandes de contrôle ont été utilisées, notamment `SHOW SLAVE STATUS`. Ces vérifications ont permis de confirmer que les threads IO et SQL de l'esclave étaient bien actifs, que la connexion au maître était établie et qu'aucune erreur de réplication n'était présente. Cette étape a été essentielle pour garantir la fiabilité du système et s'assurer que les deux conteneurs restaient parfaitement synchronisés en temps réel.

```
joseph@debian:~$ docker network create reseau-mangas
c8ca84ddaa9b49760e74b32b99c2daa36092e5c36827784d59360592cf6694aa
joseph@debian:~$ docker network connect reseau-mangas maitre
joseph@debian:~$ docker network connect reseau-mangas phpmyadmin
joseph@debian:~$ docker network inspect reseau-mangas
[
    {
        "Name": "reseau-mangas",
        "Id": "c8ca84ddaa9b49760e74b32b99c2daa36092e5c36827784d59360592cf6694aa",
        "Created": "2026-01-14T11:12:05.438803115+01:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.19.0.0/16",
                    "IPRange": "",
                    "Gateway": "172.19.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {},
        "Labels": {},
```

Voici une image qui illustre la containerisation de base de données maître esclave.

```
joseph@debian:~$ docker run -d \
  --name phpmyadmin \
  --network reseau-mangas \
  -p 8080:80 \
  -e PMA_HOST=maitre \
  -e PMA_PORT=3306 \
  phpmyadmin/phpmyadmin
docker: Error response from daemon: Conflict. The container name "/phpmyadmin" is already in use by container "6cc6bd21d7f18a51d298d1952c
febcee466c72e84283e21342ccc2b41a9c261c". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
joseph@debian:~$ docker stop phpmyadmin
docker rm phpmyadmin
phpmyadmin
phpmyadmin
joseph@debian:~$ docker run -d \
  --name phpmyadmin \
  --network reseau-mangas \
  -p 8080:80 \
  -e PMA_HOST=maitre \
  -e PMA_PORT=3306 \
  phpmyadmin/phpmyadmin
b65d43c90b17f3cb7c6a217ee363e3a8da929788d613c0a04c23c0e6384afe8b
joseph@debian:~$ docker exec -i mariadb-container2 sh -c 'mariadb -uroot -pjoseph -e "SHOW DATABASES;"'
Database
information_schema
mysql
performance_schema
sys
joseph@debian:~$ docker exec maitre sh -c 'mariadb-dump -uroot -pjoseph --all-databases' > all_databases.sql
joseph@debian:~$ docker exec -i mariadb-container2 sh -c 'mariadb -uroot -pjoseph' < all_databases.sql
joseph@debian:~$ docker exec -i mariadb-container2 sh -c 'mariadb -uroot -pjoseph -e "SHOW DATABASES;"'
Database
information_schema
```

```
MariaDB [(none)]> exit;
Bye
joseph@debian:~$ docker exec -it maitre bash
root@74d71f42b70a:/# cat > /etc/mysql/mariadb.conf.d/60-replication.cnf <<'EOF'
[mysqld]
server-id=1
log_bin=mysql-bin
binlog_format=ROW
binlog_expire_logs_seconds=604800
EOF
root@74d71f42b70a:/# exit
docker restart maitre
exit
joseph@debian:~$ docker exec -it mariadb-container2 bash
root@9ec0b169b601:/# cat > /etc/mysql/mariadb.conf.d/60-replication.cnf <<'EOF'
[mysqld]
server-id=2
relay_log=relay-bin
read_only=1
EOF
root@9ec0b169b601:/# exit
exit
joseph@debian:~$ docker restart mariadb-container2
mariadb-container2
joseph@debian:~$ docker restart maitre
maitre
joseph@debian:~$ docker exec -it maitre mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
```

## Création des conteneurs Mariadb (Maître-Esclave)

Une fois les deux conteneurs MariaDB opérationnels, la synchronisation entre le conteneur maître et le conteneur esclave a été mise en place afin de garantir la cohérence des données. Dans un premier temps, le maître a été configuré pour générer des journaux binaires (binary logs), indispensables à la réplication. De son côté, l'esclave a été paramétré pour se connecter au maître à l'aide d'un utilisateur dédié à la réplication et pour lire en continu ces journaux binaires. Concrètement, la configuration de la réplication sur l'esclave a été réalisée à l'aide de la commande `CHANGE MASTER TO`, qui permet de spécifier l'adresse du maître, le port, l'utilisateur de réplication ainsi que le fichier de log et la position à partir desquels démarrer la synchronisation. Une fois cette configuration appliquée, la réplication a été lancée avec la commande START SLAVE.

À partir de ce moment, chaque action effectuée sur la base de données du maître, comme la création d'une base, la création d'une table ou l'insertion de données était automatiquement répliquée sur l'esclave. Par exemple, lorsqu'une base de données et une table ont été créées sur le maître, celles-ci sont apparues instantanément sur l'esclave sans aucune action supplémentaire

```
MariaDB [(none)]> STOP SLAVE;
Query OK, 0 rows affected, 1 warning (0.000 sec)

MariaDB [(none)]> CHANGE MASTER TO
    ->   MASTER_HOST='maitre',
    ->   MASTER_USER='repl',
    ->   MASTER_PASSWORD='ReplPass123!',
    ->   MASTER_PORT=3306,
    ->   MASTER_LOG_FILE='mariadb-bin.000004',
    ->   MASTER_LOG_POS=786;
Query OK, 0 rows affected, 1 warning (0.015 sec)

MariaDB [(none)]> START SLAVE;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> SHOW SLAVE STATUS\G
*************************** 1. row ***************************
               Slave_IO_State: Waiting for master to send event
                  Master_Host: maitre
                  Master_User: repl
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mariadb-bin.000004
          Read_Master_Log_Pos: 786
               Relay_Log_File: relay-bin.000002
                Relay_Log_Pos: 557
        Relay_Master_Log_File: mariadb-bin.000004
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
          Replicate_Ignore_DB:
           Replicate_Do_Table:
       Replicate_Ignore_Table:
      Replicate_Wild_Do_Table:
  Replicate_Wild_Ignore_Table:
```

Pour vérifier le bon fonctionnement de la synchronisation, des commandes de contrôle ont été utilisées, notamment SHOW SLAVE STATUS\G.

Cette commande a permis de confirmer que les deux threads essentiels à la réplication étaient actifs :

- `Slave_IO_Running: Yes`, indiquant que l'esclave parvenait bien à se connecter au maître et à lire les journaux binaires ;

- `Slave_SQL_Running: Yes`, confirmant que les événements lus étaient correctement exécutés sur l'esclave.

L'indicateur `Seconds_Behind_Master` a également permis de vérifier que l'esclave était bien synchronisé en temps réel avec le maître. Cette phase de tests et de validation a été essentielle pour s'assurer de la fiabilité de la réplication et garantir que les deux conteneurs MariaDB restaient parfaitement synchronisés durant toute l'exploitation du système.

| Commande | Fonction |
|---|---|
| CREATE USER 'repl'@'%' | Créer l'utilisateur de réplica |
| GRANT REPLICATION SLAV | Donner les droits nécessaire à la réplica |
| FLUSH PRIVILEGES; | Recharge les droits immédiatement |

```
joseph@debian:~$ docker exec -it maitre mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 12.1.2-MariaDB-ubu2404-log mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> FLUSH TABLES WITH READ LOCK;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> SHOW MASTER STATUS;
+--------------------+----------+--------------+------------------+
| File               | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+--------------------+----------+--------------+------------------+
| mariadb-bin.000004 |      786 |              |                  |
+--------------------+----------+--------------+------------------+
1 row in set (0.001 sec)

MariaDB [(none)]> exit;
```

## Création des 2 bases de données et réplication

Une fois la réplication maître-esclave correctement configurée et validée, deux bases de données ont été créées directement sur le conteneur maître afin de tester le fonctionnement réel du système. Ces bases ont été accompagnées de tables et de données de test, permettant de simuler un usage concret d'une base de données en production.

La création des bases et des tables a été effectuée sur le maître à l'aide de commandes SQL classiques, par exemple :

```
MariaDB [(none)]> exit;
Bye
joseph@debian:~$ docker exec -it maitre mariadb -u root -pjoseph
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 12.1.2-MariaDB-ubu2404-log mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE IF NOT EXISTS testrep;
    nom VARCHAR(100),
    auteur VARCHAR(100),
    prix DECIMAL(6,2),
    ville VARCHAR(50),
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE testrep;
Database changed
MariaDB [testrep]>
MariaDB [testrep]> CREATE TABLE IF NOT EXISTS top_mangas (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     nom VARCHAR(100),
    ->     auteur VARCHAR(100),
    ->     prix DECIMAL(6,2),
    ->     ville VARCHAR(50),
    ->     editeur VARCHAR(100),
    ->     date_sortie DATE,
    ->     ventes INT
    -> );
Query OK, 0 rows affected (0.006 sec)
```

```
MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| testrep            |
| w3tel              |
+--------------------+
6 rows in set (0.001 sec)

MariaDB [(none)]> CREATE DATABASE Aubin;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> CREATE DATABASE Aubin;
ERROR 1007 (HY000): Can't create database 'Aubin'; database exists
MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| Aubin              |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| testrep            |
| w3tel              |
+--------------------+
7 rows in set (0.001 sec)

MariaDB [(none)]> exit;
Bye
joseph@debian:~$ docker exec -it mariadb-container2 mariadb -u root -pjoseph
```

# Projet (Kubernetes)

## Présentation

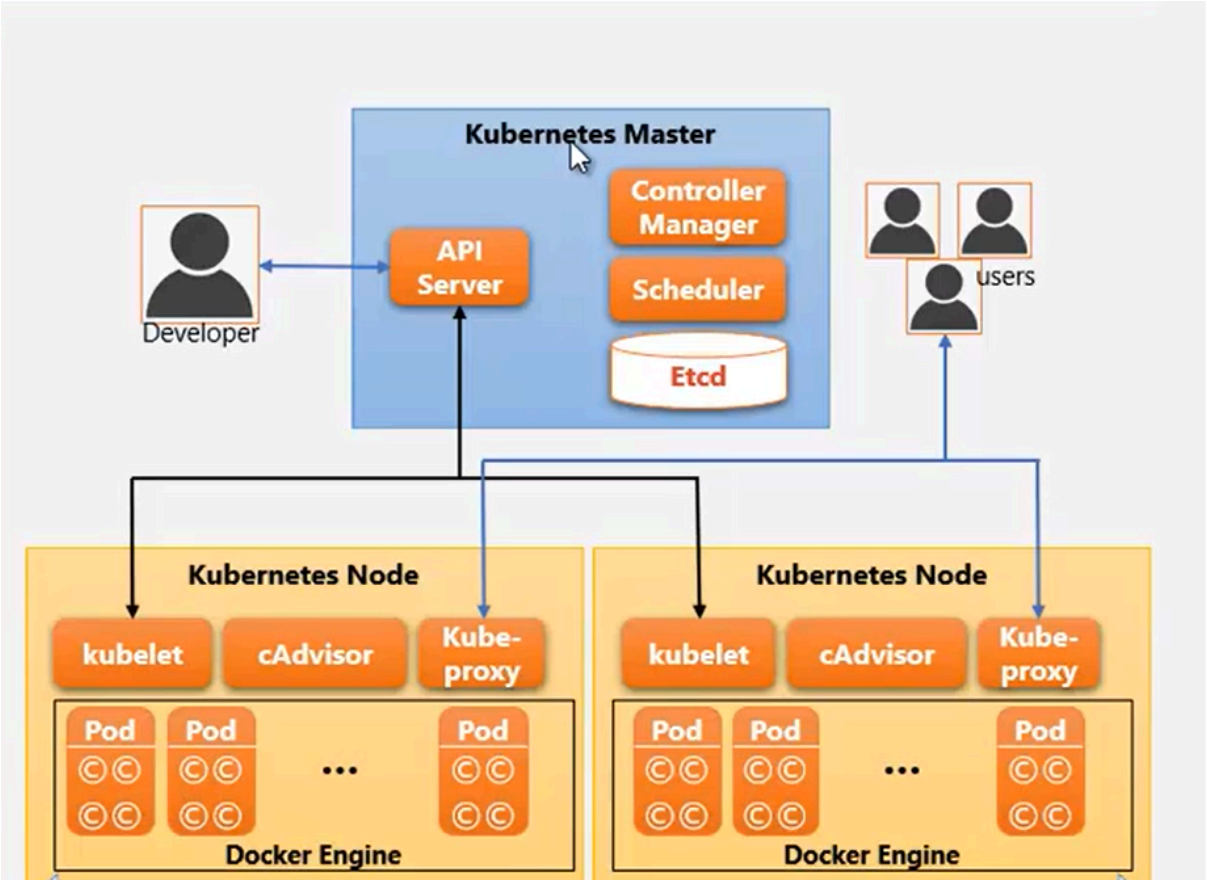
Kubernetes est une plateforme d'orchestration de conteneurs open source, initialement développée par Google, qui permet de déployer, gérer et mettre à l'échelle des applications conteneurisées de manière automatisée. Contrairement à Docker seul, qui se limite principalement à la création et à l'exécution de conteneurs, Kubernetes intervient à un niveau supérieur en coordonnant plusieurs conteneurs répartis sur un ou plusieurs nœuds.

L'objectif principal de Kubernetes est d'assurer la haute disponibilité, adaptabilité et la résilience des applications. Il permet notamment de redémarrer automatiquement des conteneurs en cas de panne, de répartir la charge entre plusieurs instances, de gérer les communications réseau entre services et de faciliter les mises à jour sans interruption de service.

Kubernetes repose sur des fichiers de configuration appelés **manifestes**, écrits en YAML, qui décrivent l'état souhaité de l'infrastructure (pods, services, déploiements,

volumes, etc.). Le cluster Kubernetes se charge ensuite d'atteindre et de maintenir cet état automatiquement. Cette approche déclarative permet une gestion plus fiable et plus reproductible des environnements.

Dans le cadre de ce projet, Kubernetes a été utilisé pour déployer et orchestrer les services MariaDB et phpMyAdmin. Cette transition depuis Docker vers Kubernetes a permis de passer d'une gestion manuelle des conteneurs à une architecture plus professionnelle, mieux adaptée aux environnements réels de production, intégrant des notions comme les services réseau, les namespaces et le reverse proxy.




## Création des manifestes (Mariadb et phpmyadmin)

Dans la seconde partie du projet, l'environnement a été migré vers Kubernetes afin de bénéficier d'une orchestration plus robuste et automatisée des services. Pour cela, plusieurs **manifestes YAML** ont été rédigés afin de déployer MariaDB et phpMyAdmin au sein du cluster.

Pour MariaDB, l'utilisation de Kubernetes a nécessité la création de ressources adaptées, notamment des **StatefulSets** et des **Services**. Le StatefulSet a été

privilégié afin de garantir une identité stable aux pods MariaDB ainsi que la persistance des données grâce à des volumes persistants. Les paramètres essentiels, comme le mot de passe root, le nom du cluster ou encore les ports d'écoute, ont été définis via des variables d'environnement et des secrets Kubernetes afin de respecter les bonnes pratiques de sécurité.

Une fois les fichiers rédigés, le déploiement a été effectué à l'aide de commandes simples, par exemple :

```
kubectl apply -f maria-galera-v1.yaml
```

Pour phpMyAdmin, un **déploiement** a été utilisé, car ce service ne nécessite pas de persistance de données locales. Le manifeste a permis de définir l'image Docker utilisée, le port HTTP exposé ainsi que les variables d'environnement nécessaires à la connexion à MariaDB. Là encore, le déploiement a été réalisé via Kubernetes :

```
kubectl apply -f phpmyadmin.yaml
```

Des commandes de vérification telles que `kubectl get pods` et `kubectl get services` ont ensuite permis de s'assurer que les pods étaient bien en état *Running* et que les services étaient correctement exposés au sein du cluster.

## Reverse proxy des manifestes (MariaDB et phpMyAdmin)

Une fois les services déployés, un mécanisme de **reverse proxy** a été mis en place à l'aide des Services Kubernetes. Des services de type **NodePort** ont été configurés afin de rendre phpMyAdmin et MariaDB accessibles depuis l'extérieur du cluster, tout en conservant une communication interne propre et sécurisée entre les composants.

phpMyAdmin a été configuré pour se connecter automatiquement à MariaDB grâce à la variable d'environnement `PMA_HOST`, qui pointe vers le service Kubernetes associé au MariaDB maître. Par exemple :

```
- name: PMA_HOST
  value: "mariadb-galera.db.svc.cluster.local"
```

Cette approche permet à phpMyAdmin de se connecter dynamiquement à la base de données sans dépendre de l'adresse IP d'un pod, ce qui est essentiel dans un environnement Kubernetes où les pods peuvent être recréés à tout moment.

L'accès à phpMyAdmin a ensuite été validé via le port exposé par le NodePort, en utilisant l'adresse IP du nœud Kubernetes et le port configuré, par exemple :

```
http://<IP_du_node>:32080
```

Grâce à cette configuration, l'administration de MariaDB a pu être réalisée via une interface web, tout en respectant l'architecture Kubernetes et les principes de séparation des services. Cette dernière étape a permis de conclure le projet en démontrant une intégration fonctionnelle entre MariaDB et phpMyAdmin dans un environnement orchestré (voir Annexes 2).

# Projet Formulaire d’inscription W3TEL

## Commandes utilisées et explication

```
PS C:\Users\admin\Inscription> cd .\.vscode\
PS C:\Users\admin\Inscription\.vscode> npm run dev

> inscription@0.0.0 dev
> vite


  VITE v7.3.1  ready in 783 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
10:09:11 [vite] (client) hmr update /src/views/Login.vue, /src/base.css
10:12:20 [vite] (client) hmr update /src/views/Register.vue, /src/base.css
10:27:03 [vite] (client) hmr update /src/base.css
10:27:15 [vite] (client) hmr update /src/views/Dashboard.vue, /src/base.css
```

| | |
|---|---|
| node server.cjs | Lancer ton backend Node/Express sur le port 3000 |
| npm install | installer toutes les dépendances du projet front |
| npm run dev | Démarrer le serveur de dev Vue |
| npm install express mysql2 cors bcrypt | Commande permettant d’installer plusieurs bibliothèques essentielles :<br>• **Express** : framework web facilitant la création de serveurs et d’API REST<br>• **mysql2** : bibliothèque permettant la connexion et les échanges avec une base de données MySQL ou MariaDB<br>• **cors** : middleware gérant les autorisations de requêtes entre différentes origines (sécurité web) |

| | |
|---|---|
| | • **bcrypt** : outil de chiffrement utilisé pour sécuriser les mots de passe des utilisateurs |
| npm install mysql2@latest | Commande utilisée pour installer la version la plus récente de la bibliothèque `mysql2`.<br>Elle permet de bénéficier des dernières améliorations, corrections et mises à jour de sécurité |

## Démarche de conception (partie front-end)

Le projet s’est déroulé en plusieurs phases :

● **Phase 1** : Maquettage (structure, choix de couleurs et typos, arborescence)

● **Phase 2** : Développement avec Vue.js, création des composants

● **Phase 3** : Intégration des contenus (texte, projets, liens, images)

● **Phase 4** : Ajout des animations et effets dynamiques (Framer Motion, défilement fluide)

● **Phase 5** : Débogage, test responsive sur plusieurs tailles d’écran

● **Phase 6**: Déploiement du site sur Docker et Kubernetes.
Malheureusement la phase 5 c'était pas à moi de m’en occuper.

| ***Nom du dossier*** | ***Son utilité*** |
|---|---|
| ***node_modules/*** | ***Contient un bout de code pour le lancement du projet.*** |
| ***src/components*** | ***Permet d’organiser la structure du code en fonction des composants.*** |
| ***backen, server.js, env*** | ***Contient des paquets indispensables pour le déploiement de l’API.*** |

## Création de formulaire avec Vue.js

Pour la création de formulaire d'inscription je repris le formulaire du projet GSH Sociale fait en SLAM et je essayer de l'adapter au demande mais je modifiais le thème du formulaire pour le faire correspond à mon projet, le but était surtout que le formulaire soit connecté aux bases de données de l'administrateur réseau (mon tuteur de stage), ainsi il peut gérer la connection des utilisateurs par des requêtes. Ensuite je mis en place un Dashboard.vue dans lequel il figure le nom de la dernière personne connecté il y a 24 heures.
Le projet consiste en la réalisation d'une application web complète dédiée à la gestion sécurisée de connexions SSH à distance, accessible depuis une interface web moderne. L'objectif principal est de proposer une solution centralisée permettant aux utilisateurs authentifiés de gérer leurs serveurs sans dépendre d'outils locaux spécifiques, tout en garantissant un haut niveau de sécurité, de fiabilité et de simplicité d'utilisation. L'application a été pensée dès le départ comme un outil professionnel, capable de s'intégrer dans un environnement réel d'administration système, avec une attention particulière portée à la protection des données sensibles et à la cohérence de l'expérience utilisateur (voir Annexes 3).

voici la ligne de code qui fait la connexion entre mon frontend et la base de donnée donc le backend.

```
const db = mysql.createConnection({
  host: "172.16.100.3",
  port: 32006,
  user: "root",
  password: "[illegible]",
  database: "w3tel_db"
});
```

au début j'avais du mal à connecter la partie front et la partie back car lors de l'installation de node.js il fallait cocher de coche certaines case mais au bout du compte j'ai pu lancer le node server.cjs (voir Annexes 3).

```
found 0 vulnerabilities
PS C:\Users\admin\Inscription\backend> npm list --depth=0
backend@1.0.0 C:\Users\admin\Inscription\backend
├── bcrypt@6.0.0
├── cors@2.8.6
├── express@5.2.1
└── mysql2@3.16.1
```

```
PS C:\Users\admin\Inscription> node server.cjs
✅ .env chargé !
🚀 Backend W3TEL : http://localhost:3000
✅ MySQL connecté
```

et pour finir je passe beaucoup de temps sur le lancement de tailwind car la commande que je tapais était pas pour la version récente de tailwind, donc forcément quand je lance l'application il prenait pas en compte le **templete**.

```
PS C:\Users\admin\Inscription> cd .\.vscode\
PS C:\Users\admin\Inscription\.vscode> npm list tailwindcss
inscription@0.0.0 C:\Users\admin\Inscription
└── tailwindcss@4.1.18

PS C:\Users\admin\Inscription\.vscode> npm install tailwindcss @tailwindcss/vite

added 11 packages, and audited 249 packages in 3s

65 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\admin\Inscription\.vscode>
```

## Architecture générale de l'application

L'architecture est divisée en trois couches principales :

1. **Frontend** : application monopage (SPA) développée avec Vue.js

2. **Backend** : API REST développée avec Node.js et Express

3. **Base de données** : MySQL pour le stockage persistant des données

Cette séparation permet d'isoler les responsabilités :

- le frontend se concentre sur l'expérience utilisateur et l'affichage,

- le backend gère la logique métier, la sécurité et les accès aux données,
- La base de données garantit la persistance et l'intégrité des informations.

## Authentification et sécurité

L'authentification est basée sur les JSON Web Tokens (JWT). Lorsqu'un utilisateur se connecte, le backend vérifie ses identifiants, met à jour la date de dernière connexion, puis génère un token sécurisé :

```
  const token = jwt.sign(
    { id: user.id, nom: user.nom, email: user.email },
    JWT_SECRET,
    { expiresIn: "24h" }
  );

  res.json({ token });
});
```

Ce token contient uniquement les informations strictement nécessaires à l'identification de l'utilisateur et possède une durée de validité limitée. Ce choix permet d'éviter la gestion de sessions côté serveur tout en garantissant un accès sécurisé aux routes protégées.

Un middleware Express vérifie systématiquement la validité du token avant d'autoriser l'accès à une route sensible.

```
const authenticateToken = (req, res, next) => {
  const auth = req.headers.authorization;
  const token = auth && auth.split(" ")[1];

  if (!token) return res.status(401).json({ message: "Token requis" });

  try {
    req.user = jwt.verify(token, JWT_SECRET);
    next();
  } catch {
    res.status(403).json({ message: "Token invalide" });
  }
};
```

Ce mécanisme garantit que seules les personnes authentifiées peuvent accéder aux fonctionnalités critiques de l'application.

## Gestion des utilisateurs et suivi des connexions

Le projet accorde une importance particulière au suivi des connexions utilisateur. À chaque connexion réussie, la date et l'heure sont enregistrées en base de données.

```
await pool.execute(
  "UPDATE users SET last_connection = NOW() WHERE id = ?",
  [user.id]
);
```

Cette information est ensuite exploitée via une route dédiée `/api/me` dans le tableau de bord afin d'afficher à l'utilisateur sa dernière activité de connexion. Le backend formate cette donnée avant de l'envoyer au frontend, ce qui garantit un affichage lisible et cohérent, par exemple

```
    lastConnFormatted = new Date(dateStr).toLocaleString("fr-FR", {
      year: "numeric",
      month: "2-digit",
      day: "2-digit",
      hour: "2-digit",
      minute: "2-digit",
      second: "2-digit",
      hour12: false,
    });
  }

  res.json({
    nom: user.nom,
    email: user.email,
    lastConnection: lastConnFormatted,
  });
```

Ce choix améliore l'expérience utilisateur en apportant une information claire et utile directement sur le tableau de bord.

## Tableau de bord (Dashboard.vue)

Le tableau de bord représente le cœur fonctionnel de l'application. Il s'agit du premier écran accessible après authentification et il centralise toutes les informations importantes : identité de l'utilisateur, date de dernière connexion, statistiques générales et liste des serveurs SSH enregistrés. Le chargement des données

s'effectue de manière asynchrone au montage du composant Vue, ce qui permet d'éviter tout blocage de l'interface et d'améliorer la perception de performance.

```
onMounted(() => {
  loadCurrentUser();
  loadServers();
});
</script>
```

La récupération des informations utilisateur se fait de manière asynchrone, en respectant le principe de séparation des responsabilités.

```
const loadCurrentUser = async () => {
  try {
    const res = await fetch("http://localhost:3000/api/me", {
      headers: authHeaders()
    });

    if (!res.ok) throw new Error("Non autorisé");

    const data = await res.json();
    currentUser.value.nom = data.nom || "Utilisateur";
    lastConnection.value = data.lastConnection || "–";

  } catch (err) {
    console.error("Erreur utilisateur :", err);
  }
};
```

Les informations sont affichées sous forme de cartes et d'éléments visuels clairs, avec une attention particulière portée à l'esthétique générale, afin de proposer une expérience utilisateur moderne et agréable (voir Annexes 3).

## Gestion des serveurs SSH

La gestion des serveurs SSH constitue une autre fonctionnalité clé du projet. Chaque utilisateur peut ajouter, consulter et supprimer ses propres serveurs. Les informations nécessaires à la connexion (hôte, port, utilisateur SSH, méthode d'authentification) sont stockées en base de données et associées à l'utilisateur connecté. Les données sensibles, telles que les mots de passe SSH, sont chiffrées avant d'être enregistrées, ce qui permet de limiter les risques en cas de compromission de la base de données. Le frontend offre une interface simple et intuitive pour gérer ces serveurs, avec des actions clairement identifiées, comme la connexion ou la suppression.

```
const [result] = await pool.execute(
  `INSERT INTO servers (name, host, port, ssh_user, ssh_password, ssh_auth, is_active)
   VALUES (?, ?, ?, ?, ?, ?, 1)`,
  [name, host, port || 22, ssh_user, encryptedPassword, authMethod]
);
```

## Terminal SSH en temps réel

L'une des fonctionnalités les plus avancées du projet est l'intégration d'un terminal SSH en temps réel directement dans le navigateur.

```
// Connexion WebSocket SSH
const connectWebSocket = () => {
  const token = localStorage.getItem('token')
  if (!token) return router.push('/login')

  ws.value = new WebSocket(`ws://localhost:3000/ws/ssh?serverId=${serverId}&token=${token}`)

  ws.value.onopen = () => {
    term.value?.writeln('✅ Connexion SSH établie')
  }

  ws.value.onmessage = (e) => {
    term.value?.write(e.data)
  }

  ws.value.onclose = () => {
    term.value?.writeln('\r\n⚠ Déconnecté du serveur')
  }

  ws.value.onerror = (err) => {
    term.value?.writeln(`\r\n❌ Erreur SSH: ${err.message || err}`)
  }
}
```

Cette fonctionnalité repose sur l'utilisation de WebSockets, permettant une communication bidirectionnelle entre le client et le serveur. Lorsqu'un utilisateur se connecte à un serveur, la connexion SSH est établie exclusivement côté backend, et les flux d'entrée et de sortie sont relayés en temps réel vers le navigateur.

Cette approche garantit que les identifiants SSH ne transitent jamais directement côté client, ce qui renforce considérablement la sécurité de l'ensemble du système. Le terminal offre ainsi une expérience proche d'un accès SSH classique, tout en restant entièrement web (voir Annexes 3) .

## Gestion de la navigation et des accès

Enfin, la navigation au sein de l'application est strictement contrôlée afin d'éviter toute incohérence dans le parcours utilisateur. Grâce à Vue Router et à un guide global, les routes protégées sont inaccessibles sans authentification, et les pages publiques comme l'accueil, la connexion ou l'inscription sont automatiquement bloquées dès lors que l'utilisateur est connecté. Ce mécanisme assure une logique de navigation claire et évite les erreurs de manipulation ou les accès non souhaités.

```
router.beforeEach((to, from, next) => {
  const token = localStorage.getItem("token");

  // 🔒 Accès interdit aux routes protégées sans token
  if (to.meta.requiresAuth && !token) {
    return next({ name: "Login" });
  }

  // 🚫 Si déjà connecté, interdiction d'accéder à Home / Login / Register
  if (
    token &&
    (to.name === "Home" ||
      to.name === "Login" ||
      to.name === "Register")
  ) {
    return next({ name: "Dashboard" });
  }

  // ✅ Navigation autorisée
  next();
});

export default router;
```

.

# Conclusion

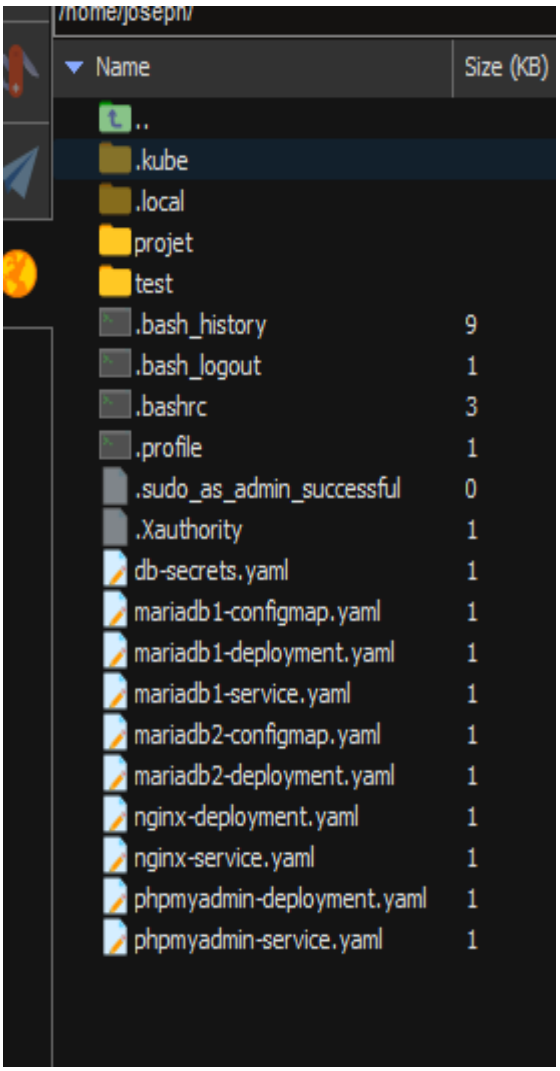
En conclusion, j’ai effectué des semaines de stage dans l’entreprise W3TEL. Grâce à ce stage, j’ai pu découvrir le monde professionnel dans un autre type d’organisation que dans une entreprise. Cela m’a permis de me rendre compte de comment est vraiment le milieu professionnel, quelles sont les exigences pour la création du site web, la gestion de réseaux pour un développeur et comment cette mise en place est effectuée.

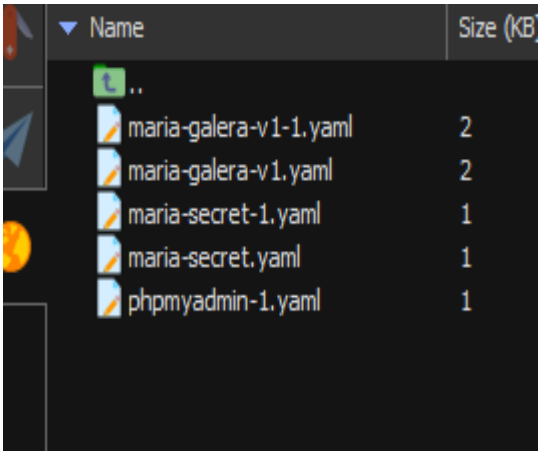
Effectuer ce stage permet d'acquérir une nouvelle compétence qui implique d'effectuer plusieurs tâches en même temps : c'est le fait d'être multitâche. Cette dernière m'a permis de m'y habituer.

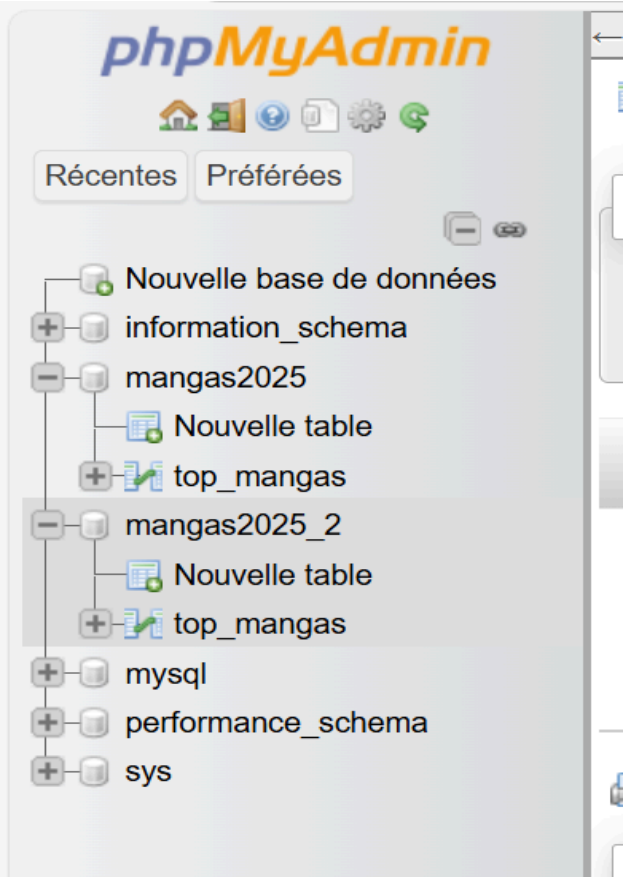
Je remercie notamment Aubin KUSONDAMA de m’avoir encouragé et aidé dans l’apprentissage des dockers, kubernetes et base de données (le fait de toujours appliquer la stratégie de moindre privilège) et également de m’avoir donné des conseils. Un grand merci à lui pour tout, du début et à la fin tous ainsi que les membres du personnel du W3TEL de m'avoir accueilli, je pense en particulier au chef de production Cédric BORNECQUE de m’avoir encouragé pour la suite

# Annexes

## Annexes 1



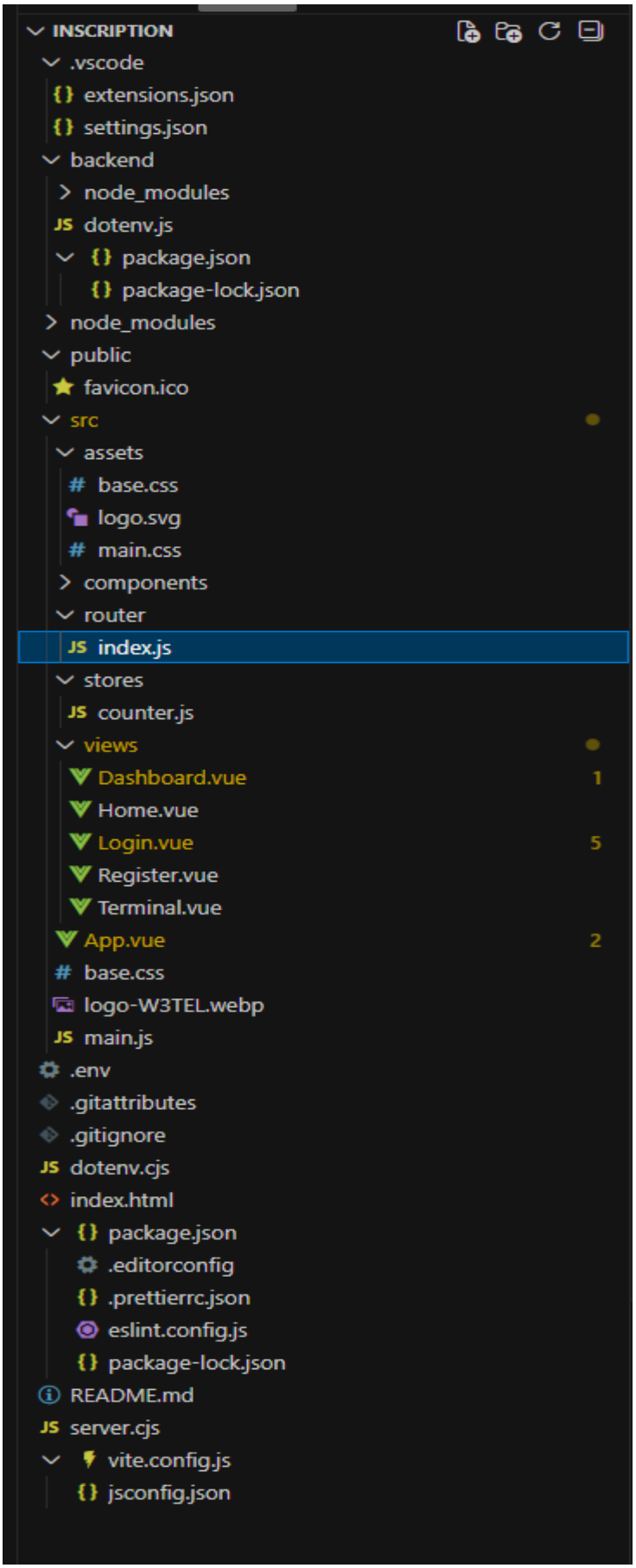



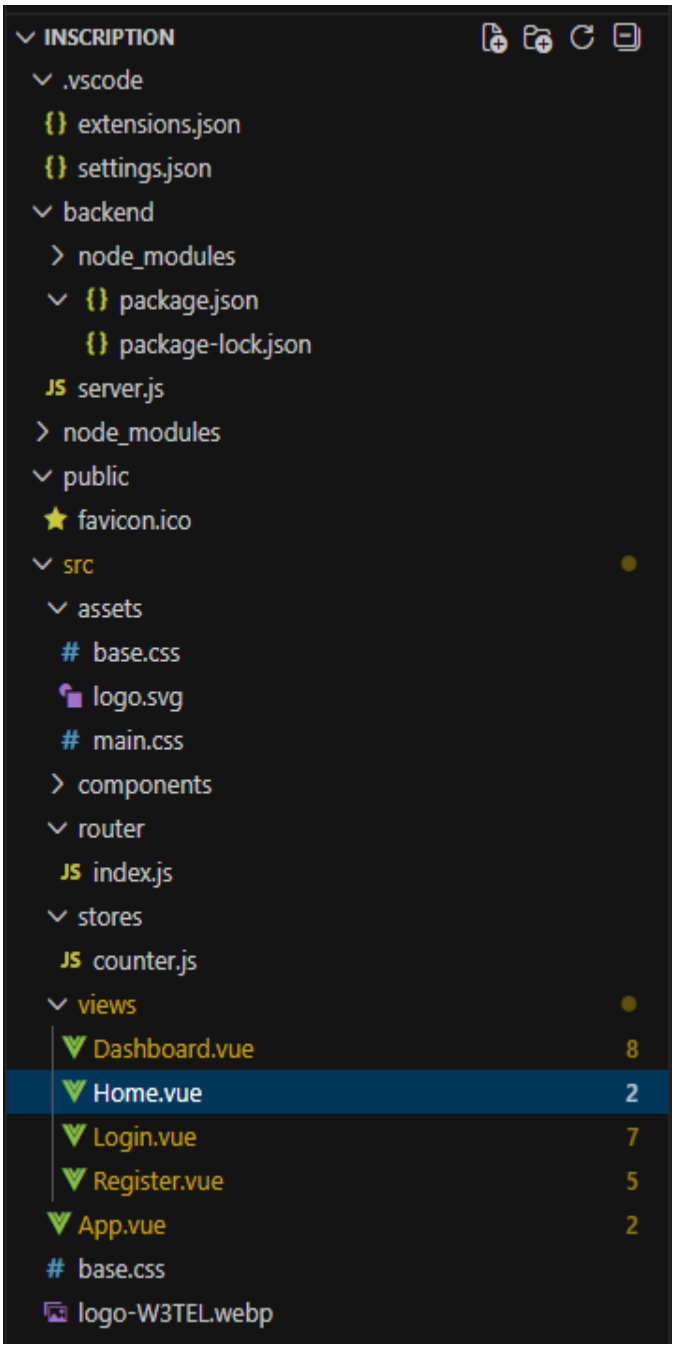



# Annexes 2 (Fin de travail Kubernetes)

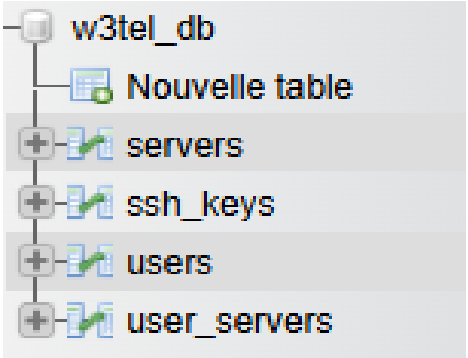
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

Your MariaDB connection id is 120
Server version: 11.8.2-MariaDB-log Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| aubin              |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| test               |
+--------------------+
6 rows in set (0.001 sec)

MariaDB [(none)]> exit
Bye
I have no name!@mariadb-galera-1:/$ exit
exit
root@master:/home/joseph/projet/version-2# kubectl exec -it mariadb-galera-2 -n db -- bash
I have no name!@mariadb-galera-2:/$ mysql -u root -p
mysql: Deprecated program name. It will be removed in a future release, use '/opt/bitnami/mariadb/bin/mariadb' instead
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 119
Server version: 11.8.2-MariaDB-log Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| aubin              |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| test               |
+--------------------+
6 rows in set (0.001 sec)

MariaDB [(none)]> exit
Bye
I have no name!@mariadb-galera-2:/$ exit
exit
root@master:/home/joseph/projet/version-2# kubectl get svc -n db
NAME                     TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)                          AGE
mariadb-galera           ClusterIP   10.43.236.9   <none>        3306/TCP                         12m
mariadb-galera-headless  ClusterIP   None          <none>        3306/TCP,4567/TCP,4568/TCP,4444/TCP   12m
mariadb-galera-nodeport  NodePort    10.43.86.41   <none>        3306:32006/TCP                   12m
phpmyadmin               NodePort    10.43.58.94   <none>        80:32080/TCP                     8m6s
root@master:/home/joseph/projet/version-2# kubectl exec -it mariadb-galera-2 -n db -- bash
I have no name!@mariadb-galera-2:/$ mysql -u root -p
mysql: Deprecated program name. It will be removed in a future release, use '/opt/bitnami/mariadb/bin/mariadb' instead
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 174
Server version: 11.8.2-MariaDB-log Source distribution
```

## Annexes 3 (partie front-end)




## Annexes 3 (partie back-end)



# Glossaire

*Ce dictionnaire contient un ensemble de termes techniques afin de mieux comprendre ses significations.*

| | |
|---|---|
| Docker | Plateforme permettant de créer, déployer et exécuter des applications dans des conteneurs. Chaque conteneur embarqué l'application et ses dépendances, garantissant un fonctionnement identique sur tous les environnements |
| Conteneur | Environnement isolé dans lequel une application s'exécute. Il est plus léger qu'une machine virtuelle et démarre très rapidement |
| MariaDB | Système de gestion de base de données relationnelle, compatible avec MySQL, utilisé pour stocker et gérer les données du projet |
| phpMyAdmin | interface web permettant d'administrer une base de données MariaDB/MySQL (création de bases, tables, utilisateurs, requêtes SQL, etc.) |
| Maître / Esclave (Master / Slave) | Architecture de réplication dans laquelle le serveur maître gère les écritures, tandis que l'esclave réplique automatiquement les données pour assurer la disponibilité et la cohérence |
| Réplication | Mécanisme permettant de copier automatiquement les données d'une base de données vers une autre afin de garantir la synchronisation |
| Binlog (Binary Log) | Journal binaire généré par MariaDB/MySQL qui enregistre |

| | |
|---|---|
| | toutes les modifications de données effectuées sur le serveur maître |
| Kubernetes | Plateforme d'orchestration de conteneurs permettant de déployer, gérer et mettre à l'échelle des applications automatiquement |
| Cluster Kubernetes | Ensemble de machines (nœuds) sur lesquelles Kubernetes exécute les conteneurs |
| Node (nœud) | Machine physique ou virtuelle faisant partie du cluster Kubernetes |
| Pod | Plus petit serveur déployable dans Kubernetes. Il peut contenir un ou plusieurs conteneurs partageant le même réseau et stockage |
| Deployment | Ressource Kubernetes permettant de gérer des pods stateless (sans données persistantes), comme phpMyAdmin |
| StatefulSet | Ressource Kubernetes utilisée pour les applications nécessitant une identité stable et un stockage persistant, comme MariaDB |
| ClusterIP | Type de Service accessible uniquement à l'intérieur du cluster Kubernetes |
| NodePort | Type de Service exposant une application sur un port fixe du nœud Kubernetes, accessible depuis l'extérieur |
| YAML | Langage de configuration utilisé par Kubernetes pour décrire les ressources (Pods, Services, Déployments, etc.) |
| Manifeste Kubernetes | Fichier YAML décrivant les ressources à créer ou gérer dans le cluster |

| | |
|---|---|
| Namespace | Espace logique permettant d'isoler et d'organiser les ressources Kubernetes (exemple : db). |
| Secret | Objet Kubernetes permettant de stocker des informations sensibles (mots de passe, clés) de manière sécurisée |
| kubectl | Outil en ligne de commande pour interagir avec Kubernetes |
| kubectl apply -f fichier.yaml | Crée ou met à jour des ressources Kubernetes à partir d'un manifeste YAML |
| kubectl get pods | Affiche la liste des pods et leur état. |
| kubectl exec | Permet d'exécuter une commande directement à l'intérieur d'un conteneur |
| kubectl describe pod | Affiche des informations détaillées sur un pod (état, erreurs, événements) |
| API (Application Programming Interface) | Une API est un ensemble de règles qui permet à deux applications de communiquer entre elles. Dans ce projet, l'API développée avec Express sert d'intermédiaire entre le frontend (Vue.js) et la base de données. Le frontend envoie des requêtes (par exemple pour se connecter ou récupérer les informations de l'utilisateur) et l'API renvoie les réponses correspondantes |
| JWT (JSON Web Token) | Le JWT est un mécanisme d'authentification basé sur des jetons chiffrés. Après une connexion réussie, le serveur génère un token contenant l'identifiant de l'utilisateur |
| Middleware | Un middleware est une fonction |

| | |
|---|---|
| | intermédiaire exécutée avant l'accès à une route. Dans ce projet, un middleware JWT vérifie la validité du token avant d'autoriser l'accès aux routes protégées comme le Dashboard ou le Terminal |
| Backend | Le backend correspond à la partie serveur de l'application. Il gère la logique métier, l'authentification, la communication avec la base de données et les connexions SSH. |
| Frontend | Le frontend est la partie visible par l'utilisateur. |
| Chiffrement (AES) | Le chiffrement est un procédé qui permet de rendre des données illisibles sans clé de déchiffrement |
| WebSocket | Les WebSockets permettent une communication en temps réel entre le client et le serveur. Ils sont utilisés pour le terminal SSH afin de transmettre instantanément les commandes et les réponses. |
| SSH (Secure Shell) | SSH est un protocole réseau sécurisé permettant de se connecter à distance à un serveur. Dans ce projet, il est utilisé pour permettre aux utilisateurs d'administrer leurs serveurs directement depuis le navigateur |